

Week 3 · 실습 워크북: 로컬 state를 S3로 옮깁니다

참고

이 문서는 실습 파트(45분)입니다. 개념은 같은 `lecture/` 폴더의 `개념워크북` 을 먼저 보세요. 여기서는 번호만 가리킵니다. `[실습]` 은 직접 입력하고, `[확인]` 은 눈으로 대조하고, `[함정]` 은 미리 피해야 할 실수이고, `[관찰 🕒]` 은 빈칸을 채웁니다. (심화 🌟) 는 먼저 끝낸 사람만 합니다.

작업 폴더는 `practice/` 입니다. `solution/` 은 막혔을 때 답을 보는 곳입니다. 위에서 아래로 그대로 따라가면 끝납니다.

오늘 만들 것

스택 2개, 리소스 12개입니다. 그리고 오늘 다루는 것은 그 리소스가 아니라 `terraform.tfstate` 파일 하나가 어디에 있는지입니다.

스택	만드는 것	개수	이 스택의 state는
<code>practice/bootstrap/</code>	S3 버킷 + 버전 관리 + 암호화 + 퍼블릭 차단 + DynamoDB 테이블	5	로컬 (오늘도, 7주차까지도)
<code>practice/app/</code>	VPC + subnet + IGW + route table + 연결 + SG + EC2	7	로컬에서 S3로 이전

비용·보안 경고

비용. app 스택을 apply하면 시간당 \$0.019가 발생합니다. destroy를 잊고 한 달 두면 약 \$13.87입니다. Block C는 선택이 아닙니다.

올리면 안 되는 값. 본인 공인 IP · AWS 계정번호 12자리 · 액세스 키 · `terraform.tfvars` · `terraform.tfstate` · `state.json` · `.pem` 파일. 마스킹 명령은 C-4에 있습니다.

체크포인트

작업 폴더가 오늘은 둘입니다. `practice/bootstrap/` 과 `practice/app/` . 오늘 자주 발생하는 사고가 다른 폴더에서 명령을 실행하는 것입니다. 이 문서의 코드블록 첫 줄에 붙은 `# pwd == ...` 주석은 그 블록의 명령을 모두 실행한 뒤에 있어야 하는 위치입니다. 블록 안에 `cd` 가 있으면 시작 위치는 그 위 블록의 값입니다. 매 블록 끝에 `pwd` 로 대조하세요.

파일	어느 스택	오늘 수정하나
<code>bootstrap/versions.tf</code> · <code>providers.tf</code> · <code>variables.tf</code>	bootstrap	읽기만
<code>bootstrap/main.tf</code>	bootstrap	채운다 (TODO ①~⑥)

파일	어느 스택	오늘 수정하나
<code>bootstrap/outputs.tf</code>	bootstrap	채운다 (TODO ⑥~⑧)
<code>app/versions.tf</code> · <code>providers.tf</code> · <code>variables.tf</code> · <code>outputs.tf</code>	app	읽기만
<code>app/network.tf</code> · <code>compute.tf</code>	app	읽기만. week2 정답 코드와 같습니다
<code>app/backend.tf</code>	app	주석을 풀고 다섯 줄을 채운다
양쪽 <code>example.tfvars</code>	둘 다	<code>terraform.tfvars</code> 로 복사해 값 채우기

참고

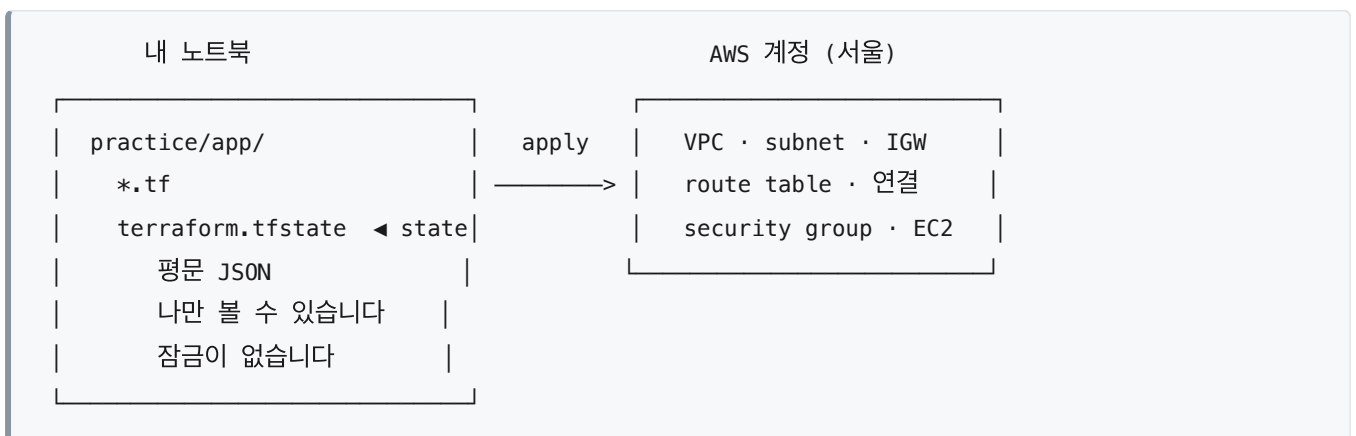
`app/network.tf` 와 `app/compute.tf` 는 이미 완성돼 있습니다. week2에서 여러분이 직접 채운 그 코드입니다. 오늘 배울 것은 그 state가 어디 있는가이므로, 같은 코드를 두 번 입력하게 하지 않습니다. 코드에 적어둔 주의 사항 설명은 개념워크북 Part 5에서 읽어봅니다.

오늘의 지도: state가 사는 곳

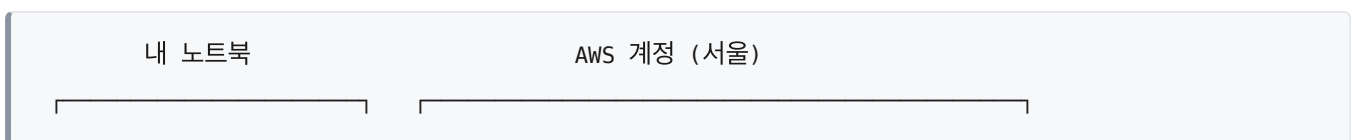
week1에서는 명령의 흐름을, week2에서는 리소스 사이의 참조를 따라갔습니다. 오늘은 파일 하나가 위치하는 곳을 따라갑니다.

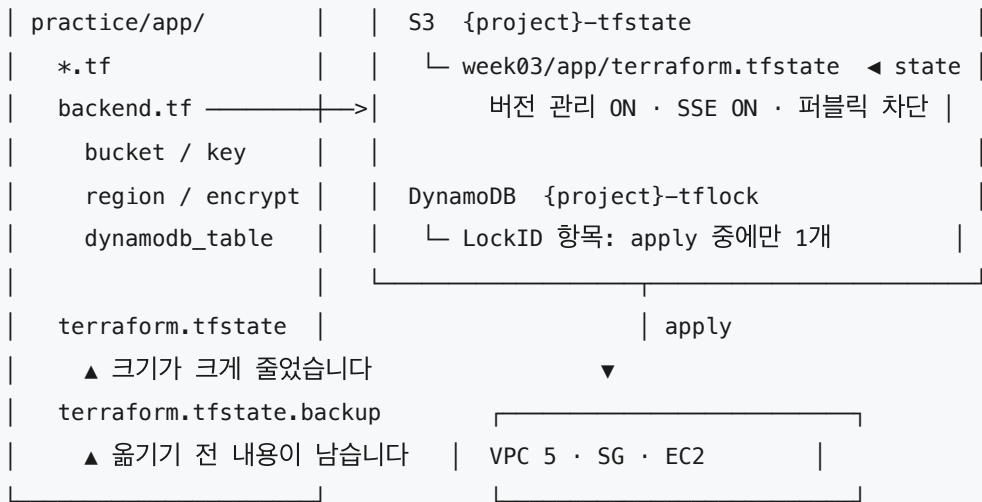
이 구성도를 한 세션에 세 번 사용합니다. Block A에서 저장소를 만들고, Block B에서 옮기고, Block C에서 역순으로 정리합니다.

① BEFORE: 오늘 시작 시점 (= week2가 끝난 상태)



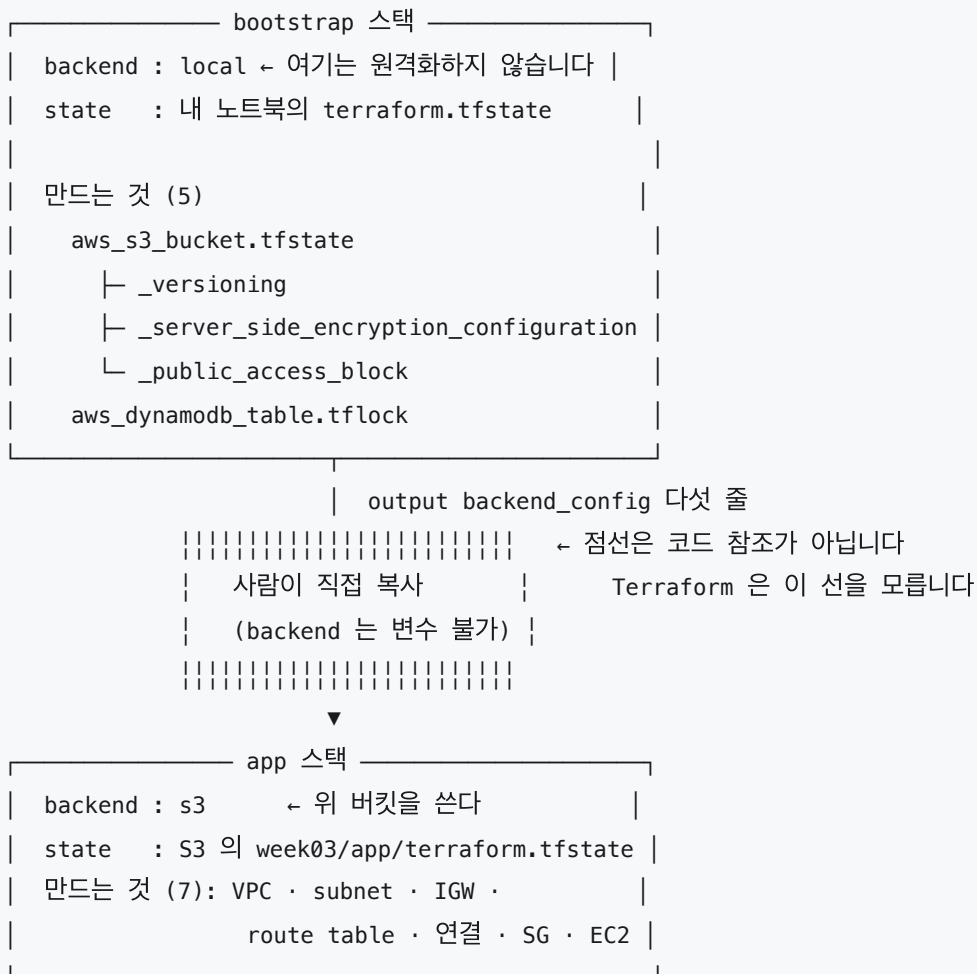
② AFTER: 오늘 끝날 때





읽는 법: 새로 생긴 참조는 하나뿐입니다. `backend.tf` 에서 S3로 향하는 것입니다. 그리고 `network.tf` 와 `compute.tf` 는 한 글자도 바뀌지 않았습니다.

③ 스택 경계선: 여기에는 참조가 없습니다



생성 순서 : bootstrap ──────────> app
 삭제 순서 : app ──────────> bootstrap (7주차)

▲
반대로 하면 state 저장소가 먼저 사라집니다.
app 을 지울 수단이 없어집니다. (C-5)

체크포인트

week2에서는 참조 11개를 잇는 것만으로 순서가 만들어졌습니다. 이 점선 위에는 참조가 없습니다. 그래서 Terraform이 순서를 계산해주지 않고, 사람이 지켜야 합니다. 그리고 스택 사이에는 `depends_on` 조차 쓸 수 없습니다.

스텝 ↔ 구성도 대응표

스텝	구성도에서 하는 일
A-3 · A-4	state 저장소를 만든다. ③의 위쪽 상자
B-2	①(BEFORE) 상태를 직접 만든다
B-4 · B-5	이전. ①에서 ②로
B-6	②의 S3 부분을 눈으로 확인
B-7	②의 DynamoDB 부분을 눈으로 확인
C-1	app 리소스만 지운다. ②의 오른쪽 아래 상자만 사라짐
C-5	③의 점선의 반대 방향으로 진행하는 절차. 7주차에만

오늘 나와야 하는 숫자

이 표와 다르면 멈추고 물어보세요. 넘어가면 뒤 단계에서 더 큰 문제가 됩니다.

지점	나와야 하는 값	다르면
A-5 bootstrap <code>plan</code>	<code>Plan: 5 to add, 0 to change, 0 to destroy.</code>	TODO ①~⑥ 확인
A-6 bootstrap <code>state list</code>	5줄	멈춤
B-2 app <code>plan</code> (로컬)	<code>Plan: 7 to add, 0 to change, 0 to destroy.</code>	멈춤
B-3 app <code>state list</code>	9줄 (리소스 7 + 데이터 소스 2)	멈춤
B-5 migrate 직후 <code>plan</code>	<code>No changes.</code>	state가 안 옮겨진 것. 멈춤
B-6 S3 객체	<code>week03/app/terraform.tfstate</code> 1개	멈춤
B-7 DynamoDB 항목	apply 중에 하나 늘고, 끝나면 다시 줄어듭니다	멈춤
C-1 <code>destroy</code>	<code>Destroy complete! Resources: 7 destroyed.</code>	멈춤

지점	나와야 하는 값	다르면
C-2 <code>state list</code>	빈 출력	멈춤

5 와 7 과 9 가 각각 다른 뜻입니다. 5는 bootstrap이 만드는 리소스, 7은 app이 만드는 리소스, 9는 app의 `state list` 줄 수입니다. `state list` 는 데이터 소스도 출력하기 때문에 9줄이 됩니다.

Block A. 워크스루 (18분): 강사와 함께 state 저장소 만들기

`practice/bootstrap/` 의 TODO 8개를 채워 리소스 5개를 만듭니다. 이 스택의 state는 로컬에 남습니다.

[확인] A-1. 사전 점검

지금 하는 일은 도구와 계정이 준비됐는지 보는 것입니다.

```
# pwd == practice/bootstrap
cd practice/bootstrap

terraform version          # v1.15.8 이상
aws sts get-caller-identity # 계정번호와 사용자 확인
```

```
Terraform v1.15.8
on darwin_arm64
```

체크포인트

`aws sts get-caller-identity` 가 실패하면 여기서 멈추세요. `aws configure` 로 자격증명을 넣지 않으면 뒤 단계를 진행할 수 없습니다. 출력된 계정번호 12자리는 화면 공유나 스크린샷에 넣지 마세요.

[실습] A-2. 내 값 채우기

지금 하는 일은 버킷 이름의 접두어가 될 `project_name` 을 정하는 것입니다.

```
# pwd == practice/bootstrap
cp example.tfvars terraform.tfvars
```

`terraform.tfvars` 를 열어 `CHANGE-ME` 를 본인 값으로 바꾸세요. 형식은 `boaz26-w3-{본인-github-id}` 입니다.

```
region      = "ap-northeast-2"
project_name = "boaz26-w3-kdh1834"
```

주의

소문자 · 숫자 · 하이픈만 쓸 수 있습니다. 대문자가 섞이면 `apply` 도중에 `InvalidBucketName` 으로 실패합니다. S3 버킷 이름 규칙이기 때문입니다. `CHANGE-ME` 를 그대로 두면 대문자 때문에 `variables.tf` 의 검사에 먼저 걸립니다.

그리고 이 값은 B-1의 app 스택에서 똑같이 써야 합니다. 지금 적어두세요.

[관찰 🍷] A-2 기록

- 내가 정한 `project_name` :

[실습] A-3. TODO ①~④: S3 버킷과 안전장치 셋

지금 하는 일은 state를 담은 버킷을 만들고, 거기에 버전 관리 · 암호화 · 퍼블릭 차단을 거는 것입니다.

`main.tf` 를 열면 주석으로 된 힌트가 있습니다. 주석을 풀어 채우세요. 왜 버킷 하나로 끝나지 않는지는 개념워크북 Part 4에 있습니다.

①은 버킷입니다.

```
resource "aws_s3_bucket" "tfstate" {
  bucket = "${var.project_name}-tfstate"

  tags = { Name = "${var.project_name}-tfstate" }
}
```

②는 버전 관리입니다. `bucket = aws_s3_bucket.tfstate.id` 가 참조이고, 이 한 줄이 버킷을 먼저 만들게 합니다.

```
resource "aws_s3_bucket_versioning" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id

  versioning_configuration {
    status = "Enabled"
  }
}
```

③은 서버측 암호화입니다.

```
resource "aws_s3_bucket_server_side_encryption_configuration" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}
```

```
}  
}
```

④는 퍼블릭 접근 차단입니다. 네 개 모두 `true` 입니다.

```
resource "aws_s3_bucket_public_access_block" "tfstate" {  
  bucket = aws_s3_bucket.tfstate.id  
  
  block_public_acls      = true  
  block_public_policy    = true  
  ignore_public_acls    = true  
  restrict_public_buckets = true  
}
```

주의

`force_destroy` 를 넣지 마세요. 켜면 `destroy` 한 번에 state 이력 전체가 사라져서 ②가 무의미해집니다.
개념워크북 13번.

[실습] A-4. TODO ⑤~⑧: 잠금 테이블과 output

지금 하는 일은 잠금용 테이블을 만들고, app 스택에 넘길 값을 output으로 꺼내는 것입니다.

⑤는 DynamoDB 테이블입니다.

```
resource "aws_dynamodb_table" "tflock" {  
  name           = "${var.project_name}-tflock"  
  billing_mode   = "PAY_PER_REQUEST"  
  hash_key      = "LockID"  
  
  attribute {  
    name = "LockID"  
    type = "S" # S = 문자열  
  }  
  
  tags = { Name = "${var.project_name}-tflock" }  
}
```

비용·보안 경고

`billing_mode` 는 `PAY_PER_REQUEST` 여야 합니다. `PROVISIONED` 로 두면 쓰지 않아도 과금됩니다.

`hash_key` 는 `LockID` 여야 합니다. Terraform이 그 이름으로 찾기 때문에, 다른 이름을 쓰면 잠금이 동작하지 않습니다.

이제 `outputs.tf` 의 ㉔ ㉕ ㉖을 채웁니다. 세 개 다 채우세요. A-6에서 ㉔의 출력을 복사해 B-4에 붙여 넣습니다.

```
output "bucket_name" {
  description = "state를 담는 S3 버킷 이름. app/backend.tf 의 bucket 에 그대로 넣습니다"
  value       = aws_s3_bucket.tfstate.id
}

output "table_name" {
  description = "잠금용 DynamoDB 테이블 이름. app/backend.tf 의 dynamodb_table 에 넣습니다"
  value       = aws_dynamodb_table.tflock.name
}

output "backend_config" {
  description = "app/backend.tf 의 backend \"s3\" 블록 안에 붙여 넣을 값 다섯 줄"
  value       = <<-EOT
    bucket      = "${aws_s3_bucket.tfstate.id}"
    key         = "week03/app/terraform.tfstate"
    region      = "${var.region}"
    dynamodb_table = "${aws_dynamodb_table.tflock.name}"
    encrypt     = true
  EOT
}
```

도움

㉔이 오늘 실습의 안전장치입니다. 직접 옮겨 적다가 버킷 이름을 틀리면 B-5의 `init` 이 `NoSuchBucket` 으로 실패합니다. 여기서는 `${}` 보간을 쓸 수 있는데 `backend` 블록 안에서는 못 쓴다는 것도 같이 보세요. 개념워크북 4번.

[확인] A-5. `fmt` → `init` → `validate` → `plan`

지금 하는 일은 `apply` 전에 코드가 맞는지 네 단계로 확인하는 것입니다.

```
# pwd == practice/bootstrap
terraform fmt
terraform init
terraform validate
terraform plan
```

`plan` 맨 아래가 이래야 합니다.

```
Plan: 5 to add, 0 to change, 0 to destroy.
```

체크포인트

5가 아니면 멈추세요. 4면 TODO 하나를 채우지 않았고, 6이면 필요 없는 리소스를 더 만들고 있습니다. A-3 · A-4로 돌아가 개수를 맞춘 다음 넘어가세요.

[함정] `plan` 이 `project_name` 을 물어보면 A-2의 `terraform.tfvars` 가 없거나 파일 이름이 틀린 것입니다. 파일 이름은 `terraform.tfvars` 여야 합니다.

[확인] A-6. apply, 그리고 다섯 줄 출력하기

지금 하는 일은 저장소를 실제로 만들고, `app` 스택에 넣을 값을 출력하는 것입니다.

```
# pwd == practice/bootstrap
terraform apply      # yes
terraform state list
```

`state list` 가 5줄이어야 합니다.

```
aws_dynamodb_table.tflock
aws_s3_bucket.tfstate
aws_s3_bucket_public_access_block.tfstate
aws_s3_bucket_server_side_encryption_configuration.tfstate
aws_s3_bucket_versioning.tfstate
```

이제 `backend`에 넣을 다섯 줄을 출력합니다.

```
# pwd == practice/bootstrap
terraform output -raw backend_config
```

```
bucket      = "boaz26-w3-kdh1834-tfstate"
key          = "week03/app/terraform.tfstate"
region      = "ap-northeast-2"
dynamodb_table = "boaz26-w3-kdh1834-tflock"
encrypt      = true
```

도움

이 출력을 그대로 복사해 두세요. 직접 옮겨 적으면 오타가 생기기 쉽습니다.

비용·보안 경고

이 폴더의 `terraform.tfstate` 를 지우지 마세요. `.gitignore` 대상이라 커밋되지 않고, 7주차에 버킷과 테이블을 지울 때 필요한 유일한 기록입니다.

[관찰 🕒] A-6 기록

- 버킷 이름:
- 테이블 이름:
- 이 스택의 state는 지금 (로컬 / S3) 에 있습니다:
- `state list` 줄 수:

Block B. 각자 진행 (20분): app을 원격으로 옮기기

`practice/app/` 의 `network.tf` 와 `compute.tf` 는 week2 정답 코드와 같습니다. 오늘은 채우지 않고 읽기만 합니다. 여기서 직접 작성하는 것은 `terraform.tfvars` 와 `backend.tf` 뿐입니다.

순서가 이 블록의 핵심입니다. 먼저 로컬 state로 만들고, 그다음 옮깁니다. 처음부터 backend를 켜면 옮길 state가 없어서 오늘의 주제를 볼 수 없습니다. 개념워크북 5번.

[실습] B-1. 내 값 채우기

지금 하는 일은 app 스택의 변수를 채우는 것입니다.

```
# pwd == practice/app
cd ../app
cp example.tfvars terraform.tfvars
curl -4 ifconfig.me      # 내 공인 IP 확인
```

`terraform.tfvars` 에서 `project_name` 은 A-2와 같은 값을 쓰고, `my_ip` 에는 위 결과를 넣습니다.

```
region      = "ap-northeast-2"
project_name = "boaz26-w3-kdh1834"
my_ip       = "1.2.3.4"
instance_type = "t3.micro"
```

주의

`my_ip` 에 `/32` 를 붙이지 마세요. 코드가 `"${var.my_ip}/32"` 로 조립하기 때문에, 이미 붙여두면 `1.2.3.4/32/32` 가 되어 검사에서 걸립니다.

그리고 `terraform.tfvars` 는 커밋 금지입니다. 본인 공인 IP가 들어 있습니다.

[확인] B-2. backend 없이 먼저 로컬로 apply

지금 하는 일은 옮길 state를 만드는 것입니다. `backend.tf` 는 아직 전체가 주석입니다. 건드리지 마세요.

```
# pwd == practice/app
terraform init      # backend 설정이 없으니 local 로 초기화됩니다
terraform plan      # "Plan: 7 to add"
terraform apply     # yes
```

```
Plan: 7 to add, 0 to change, 0 to destroy.
```

apply가 끝나면 폴더에 `terraform.tfstate` 가 생겼습니다. 지금부터 이 파일을 S3로 옮기는 것이 오늘의 실습입니다.

[관찰 🏎️] B-3. 로컬 state를 눈으로 보기

지금 하는 일은 옮기기 전에 무엇을 옮기는지 확인하는 것입니다.

```
# pwd == practice/app
terraform state list      # 9줄
ls -l terraform.tfstate  # 파일 크기
```

- `state list` 줄 수: (리소스 7 + 데이터 소스 2)
- `terraform.tfstate` 파일 크기:
- 이 파일이 지금 있는 곳은 (내 노트북 / S3) 입니다:

[실습] B-4. `backend.tf` 주석을 풀고 값 붙여넣기

지금 하는 일은 state를 어디에 둘지 코드에 적는 것입니다.

`backend.tf` 를 열어 아래쪽 주석 블록의 `#` 을 지우고, `bucket` 과 `dynamodb_table` 을 A-6의 출력으로 바꾸세요.

```
terraform {
  backend "s3" {
    bucket      = "boaz26-w3-kdh1834-tfstate"
    key         = "week03/app/terraform.tfstate"
    region      = "ap-northeast-2"
    dynamodb_table = "boaz26-w3-kdh1834-tflock"
    encrypt     = true
  }
}
```

주의

버킷 이름 한 글자만 틀려도 다음 단계의 `init` 이 실패합니다. A-6의 `terraform output -raw backend_config` 출력을 복사해 붙이세요.

[함정] 여기에 `var.project_name` 을 쓰고 싶어질 텐데 안 됩니다. `init` 은 backend를 가장 먼저 초기화하고 그 시점에는 변수가 아직 평가되지 않았습니다. 개념워크북 4번.

[확인] B-5. `init -migrate-state`

지금 하는 일은 로컬 state를 S3로 옮기는 것입니다. 오늘 실습의 핵심 명령입니다.

```
# pwd == practice/app
terraform init -migrate-state
```

Terraform이 옮길지 물어봅니다. `yes` 를 입력하세요.

```
Initializing the backend...
Do you want to copy existing state to the new backend?
Pre-existing state was found while migrating the previous "local" backend to the
newly configured "s3" backend. No existing state was found in the newly
configured "s3" backend. Do you want to copy this state to the new "s3" backend?
Enter "yes" to copy and "no" to start with an empty state.

Enter a value: yes
```

```
Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
```

`init` 중에 이런 경고가 함께 뜹니다. 정상입니다.

```
| Warning: Deprecated Parameter
|
|   on backend.tf line 7, in terraform:
|     7:     dynamodb_table = "boaz26-w3-kdh1834-tflock"
|
| The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile"
| instead.
```

line 7 과 7: 은 예시입니다. 경고는 `dynamodb_table` 이 실제로 놓인 줄을 가리키므로, `practice/app/backend.tf` 의 주석을 켜둔 상태에서는 20행으로 나옵니다.

오늘 DynamoDB 방식을 쓰는 이유는 잠금을 콘솔에서 눈으로 보기 위해서입니다. 개념워크북 9번.

옮겨졌는지 확인합니다.

```
# pwd == practice/app
terraform plan
```

No changes. Your infrastructure matches the configuration.

체크포인트

No changes. 가 아니라 7 to add 가 나오면 state가 옮겨지지 않았습니다. 새 빈 state로 시작한 것입니다. 멈추고 물어보세요. 그대로 apply 하면 리소스를 중복으로 만듭니다.

[함정] AccessDenied 가 나오면 IAM 권한 문제입니다. S3의 ListBucket · GetObject · PutObject 와 DynamoDB의 GetItem · PutItem · DeleteItem 이 필요합니다. 목록은 개념워크북 Part 3 끝에 있습니다.

[함정] NoSuchBucket 이면 B-4의 버킷 이름이 틀렸습니다. A-6 출력과 한 글자씩 대조하세요.

[확인] B-6. 로컬과 S3를 대조하기

지금 하는 일은 state가 실제로 S3에 저장됐는지 양쪽을 보는 것입니다.

```
# pwd == practice/app
ls -l terraform.tfstate*
```

로컬 terraform.tfstate 는 크기가 크게 줄었고, terraform.tfstate.backup 에 옮기기 전 내용이 남아 있습니다.

콘솔에서 S3 버킷을 열어 week03/app/terraform.tfstate 객체가 있는지 확인하세요. aws CLI가 있으면 이렇게도 됩니다.

```
# pwd == practice/app
aws s3 ls "s3://boaz26-w3-kdh1834-tfstate/week03/app/"
```

체크포인트

이 객체가 오늘 과제의 증빙입니다. 스크린샷을 지금 찍어두세요. 계정번호가 화면에 있으면 가려서 저장하세요.

[관찰 🍌] B-6 기록

- 로컬 terraform.tfstate 크기: _____
- terraform.tfstate.backup 크기: _____
- S3 객체 키 전체: _____

[관찰 🏹] B-7. 잠금 재현

지금 하는 일은 잠금이 걸리는 순간을 직접 만드는 것입니다. 터미널 두 개가 필요합니다. 둘 다 `practice/app` 입니다.

터미널 A에서 `apply`를 시작하고 `yes` 직전에 멈춰 둡니다.

```
# 터미널 A · pwd == practice/app
terraform apply      # yes 를 입력하지 말고 대기
```

터미널 B에서 같은 폴더에 `plan` 을 실행합니다.

```
# 터미널 B · pwd == practice/app
terraform plan
```

터미널 B가 이렇게 막힙니다.

```
Error: Error acquiring the state lock

Lock Info:
  ID:          ...
  Path:        boaz26-w3-kdh1834-tfstate/week03/app/terraform.tfstate
  Operation:   OperationTypeApply
  Who:         kdh1834@my-macbook
  Version:     1.15.8
  Created:     ...
```

확인이 끝나면 터미널 A에서 `Ctrl+C` 나 `no` 로 빠져나오세요.

주의

터미널 B에서는 `apply` 를 실행하지 말고 `plan` 만 실행하세요. 그리고 터미널 A를 5분 이상 방치하지 마세요. 다른 사람의 진행 속도와 맞지 않게 됩니다.

- 터미널 B에서 본 오류 첫 줄: _____
- `Lock Info` 의 `Who` 값: _____
- `Operation` 값: _____
- DynamoDB 테이블에 항목이 생겼다 사라지는 것을 봤는지: (봤다 / 못 봤다) _____
- 잠금이 왜 필요한지 한 문장으로: _____

[실습] B-8 (심화 ★). `-backend-config` 로 값 분리

지금 하는 일은 backend 값을 코드 밖 파일로 분리하는 것입니다. `backend.hcl` 은 `.gitignore` 대상입니다.

```
# pwd == practice/app
# backend.hcl 을 만들고 bucket/key/region/dynamodb_table/encrypt 를 적습니다
terraform init -backend-config=backend.hcl -reconfigure
```

backend.tf 에는 backend "s3" {} 만 남깁니다. 코드에서 계정별 값이 사라지는 것이 이 방식의 이점입니다.

주의

-reconfigure 는 state를 복사하지 않습니다. 같은 버킷과 같은 key 를 가리키므로 지금은 안전하지만, 오타가 나면 빈 state를 보게 됩니다. 실행 직후 terraform state list 가 9줄인지 반드시 확인하세요.

[실습] B-9 (심화 ★). use_lockfile 과 비교

지금 하는 일은 DynamoDB 없이 잠그는 방식과 비교하는 것입니다. Terraform 1.10 이상이 필요합니다.

dynamodb_table 줄을 지우고 use_lockfile = true 를 넣은 다음 init -reconfigure 합니다. B-5의 deprecation 경고가 사라지고, 잠금은 S3에 .tflock 객체로 생깁니다. 잠금을 콘솔에서 보기는 어려워지지만 테이블을 만들 필요가 없습니다.

Block C. 정리 (7분): destroy와 잔존 점검

비용·보안 경고

이 블록을 생략하면 EC2가 계속 켜져 있습니다. 시간당 \$0.019, 한 달이면 약 \$13.87입니다. 실습의 마지막 순서로 두지 말고 독립된 필수 블록으로 보세요.

지우는 것은 app 스택뿐입니다. bootstrap은 7주차까지 남깁니다. 반대로 하면 state 저장소가 먼저 사라져서 app을 지울 수단이 없어집니다.

[확인] C-1. app 스택 destroy

지금 하는 일은 오늘 만든 워크로드를 지우는 것입니다.

```
# pwd == practice/app
terraform destroy      # yes
```

Destroy complete! Resources: 7 destroyed.

[함정] destroy가 시큐리티 그룹이나 서브넷에서 멈추면 ENI가 남아 있는 경우입니다. 잠시 기다린 뒤 terraform destroy 를 한 번 더 실행하세요.

[확인] C-2. state가 빈 것 확인

지금 하는 일은 state가 실제로 비었는지 보는 것입니다.

```
# pwd == practice/app
terraform state list
```

빈 출력이어야 합니다. 데이터 소스도 함께 사라집니다.

[확인] C-3. 계정이 빈 것 확인

지금 하는 일은 C-2와 다른 확인입니다. `state list`가 비었다는 것은 Terraform이 관리하지 않는다는 뜻일 뿐, 계정에 아무것도 없다는 뜻이 아닙니다.

```
# pwd == week3-remote-backend
cd ../..
./scripts/check-leftover.sh
```

`Purpose=workload`는 전부 0이어야 하고, `Purpose=terraform-state`는 남아 있어야 정상입니다. 이 스크립트가 그 둘을 구분해서 보여줍니다.

[1] 오늘 지웠어야 하는 것 (Purpose=workload)

OK	EC2 인스턴스 (과금!)	0
OK	미사용 EBS 볼륨 (과금!)	0
OK	Elastic IP (과금!)	0
OK	NAT Gateway (과금!)	0
OK	workload VPC (무료·개수제한)	0

[2] 7주차까지 남겨 두는 것 (Purpose=terraform-state)

유지	state S3 버킷	boaz26-w3-kdh1834-tfstate
	↳ week03/ 객체 1개 (state가 원격에 있으면 1개)	
유지	잠금 DynamoDB 테이블	boaz26-w3-kdh1834-tflock
	↳ 항목 1개 (잠금 항목은 apply 중에만. 나머지는 state 체크섬)	

`aws` CLI가 없으면 콘솔에서 EC2 인스턴스 · EBS 볼륨 · Elastic IP · NAT Gateway 네 가지를 서울 리전에서 직접 확인하세요.

체크포인트

stop은 destroy가 아닙니다. 인스턴스를 멈추면 EC2와 퍼블릭 IPv4 요금은 멈추지만 EBS 8GiB의 월 \$0.73은 계속 발생합니다.

[관찰 🏠] C-3 기록

- Purpose=workload 항목들이 전부 0인가:
- state S3 버킷은 남아 있는가:
- DynamoDB 항목 수: apply 중 / 끝난 뒤

[실습] C-4. 제출 전 마스킹

지금 하는 일은 증빙에서 가려야 할 값을 지우는 것입니다.

```
# pwd == practice/app
cd practice/app          # C-3 에서 리포 루트로 올라왔으므로 다시 내려갑니다
ID=본인-github-id       # 예시 값 그대로 두지 마세요
echo "$ID"               # 바꿨는지 눈으로 확인
mkdir -p "../..submissions/$ID"

terraform state list > "../..submissions/$ID/state-list.txt"
```

계정번호 12자리와 IP를 가립니다. macOS와 GNU에서 `sed -i` 문법이 다르므로 파일을 새로 만드는 쪽이 안전합니다.

```
# pwd == practice/app
sed -E \
  -e 's/[0-9]{12}/<account-id>/g' \
  -e 's/([0-9]{1,3}\.){3}[0-9]{1,3}/x.x.x.x/g' \
  "../..submissions/$ID/state-list.txt" > "../..submissions/$ID/masked.txt"
mv "../..submissions/$ID/masked.txt" "../..submissions/$ID/state-list.txt"
```

커밋 직전에 확인합니다. 아무것도 안 나와야 정상입니다.

```
# pwd == week3-remote-backend
cd ../..
git diff --cached | grep -nE '([0-9]{1,3}\.){3}[0-9]{1,3}|[0-9]{12}|AKIA[0-9A-Z]{16}'
git status --porcelain | grep -E 'terraform\.tfvars|\.tfstate|\.pem'
```

비용·보안 경고

스크린샷도 마스킹하세요. 명령으로 처리되지 않습니다. 가려야 하는 것은 공인 IP와 계정번호 12자리입니다.

[개념] C-5. bootstrap 스택을 지우는 절차 (이번 주에는 실행하지 않습니다)

지금 읽는 것은 7주차에 쓸 절차입니다. 이번 주에는 실행하지 않습니다.

주의

지금 실행하면 4주차부터 7주차까지 쓸 state 저장소가 사라집니다. 이 절에는 7주차에 쓸 절차만 적어둡니다. 읽고 넘어가세요.

`force_destroy` 를 끄고 버전 관리를 켜 뒀기 때문에 `terraform destroy` 만으로는 `BucketNotEmpty` 로 실패합니다. `aws s3 rm --recursive` 로도 구버전이 남습니다. 방법이 두 가지입니다.

Terraform으로 하는 쪽이 간단합니다. `aws_s3_bucket.tfstate` 에 `force_destroy = true` 를 넣고 `apply` 한 다음 `destroy` 합니다.

```
# 7주차에만. bootstrap/main.tf 에 force_destroy = true 추가 후
terraform apply
terraform destroy
```

CLI로 직접 비우려면 버전과 삭제 마커를 각각 지워야 합니다.

```
# 7주차에만
B="boaz26-w3-kdh1834-tfstate"

aws s3api list-object-versions --bucket "$B" \
  --query '{Objects: Versions[].{Key:Key,VersionId:VersionId}}' \
  --output json > versions.json
aws s3api delete-objects --bucket "$B" --delete file://versions.json

aws s3api list-object-versions --bucket "$B" \
  --query '{Objects: DeleteMarkers[].{Key:Key,VersionId:VersionId}}' \
  --output json > markers.json
aws s3api delete-objects --bucket "$B" --delete file://markers.json

terraform destroy
```

목록이 비어 있으면 `{"Objects": null}` 이 나오고 `delete-objects` 가 인자 검증에서 실패합니다. 그 경우 그 호출은 건너뛰세요. `delete-objects` 는 한 번에 1000개까지 받습니다.

세션 후: 실습 결과 제출 (브랜치 → PR)

세션 중에는 `practice/` 에서 실습하면 됩니다. 세션이 끝나면 아래 절차를 진행하세요.

```
# pwd == week3-remote-backend
ID=본인-github-id          # {github-id} 는 셸이 치환해주지 않습니다

git switch main && git pull
```

```
git switch -c "week3/$ID"

mkdir -p "submissions/$ID/bootstrap" "submissions/$ID/app"
cp practice/bootstrap/*.tf "submissions/$ID/bootstrap/"
cp practice/bootstrap/.terraform.lock.hcl "submissions/$ID/bootstrap/"
cp practice/app/*.tf "submissions/$ID/app/" # backend.tf 포함
cp practice/app/.terraform.lock.hcl "submissions/$ID/app/"
# C-4에서 저장한 state-list.txt 는 이미 그 폴더에 있습니다 (마스킹 확인!)
# 관찰 🏹 답안을 submissions/$ID/observations.md 로 정리

git add "submissions/$ID"
git status # tfvars / tfstate / backend.hcl 안 올라갔는지!
git commit -m "week3: $ID 실습 제출"
git push -u origin "week3/$ID"
```

넣을 것	넣지 말 것
bootstrap/ · app/ 의 .tf 파일	terraform.tfvars
example.tfvars (값은 채우지 않은 상태)	terraform.tfstate · *.backup · state.json
state-list.txt (마스킹 완료)	backend.hcl
S3 객체 스크린샷 (마스킹 완료)	.pem 파일
observations.md (관찰 🏹 답안)	공인 IP · 계정번호 12자리

복사가 끝나면 `practice/` 는 되돌려 둡니다.

```
# pwd == week3-remote-backend
git restore practice/ # 복사를 끝낸 뒤에만 실행. 스켈레톤 상태로 되돌아갑니다
rm -f practice/app/backend.hcl # 내가 만든 파일은 git 이 추적하지 않습니다
```

그다음 GitHub에서 `week3/{github-id}` 에서 `main` 으로 PR을 올리면 강사가 리뷰 후 머지합니다.

체크포인트

제출은 `submissions/{github-id}/` 본인 폴더에 하세요. `practice/` 를 직접 고쳐서 올리면 머지하는 순간 다음 사람이 풀 # TODO 가 사라집니다.

체크포인트 (Definition of Done)

- ☐ A-6: bootstrap state list 5줄
- ☐ B-2: app plan 이 7 to add

- ☐ B-3: app `state list` 9줄
- ☐ B-5: `init -migrate-state` 성공, `plan` 이 `No changes.`
- ☐ B-6: S3에 `week03/app/terraform.tfstate` 객체 확인
- ☐ B-7: 터미널 두 개로 잠금 오류 재현, `Lock Info` 의 `Who` 기록
- ☐ C-1: app `destroy` 완료 (`7 destroyed`)
- ☐ C-2: `state list` 빈 출력
- ☐ C-3: `Purpose=workload` 전부 0
- ☐ C-4: 마스킹 확인
- ☐ bootstrap은 지우지 않았습니다

오늘의 명령 요약

state가 위치하는 곳이 바뀌는 순서대로 정리한 것입니다.

```
# Block A: state 저장소 (practice/bootstrap)
cp example.tfvars terraform.tfvars
terraform fmt && terraform init && terraform validate
terraform plan                # Plan: 5 to add
terraform apply                # yes
terraform output -raw backend_config # 다섯 줄 복사

# Block B: app 스택 (practice/app)
cp example.tfvars terraform.tfvars
terraform init && terraform plan    # Plan: 7 to add
terraform apply                    # yes. state 는 아직 로컬
terraform state list               # 9줄
# backend.tf 주석 풀고 위 다섯 줄 붙여넣기
terraform init -migrate-state      # yes
terraform plan                     # No changes.
                                   # 터미널 2개로 잠금 재현 (B-7)

# Block C: 정리 (practice/app)
terraform destroy                  # yes. 7 destroyed
terraform state list               # 빈 출력
../scripts/check-leftover.sh      # workload 전부 0
```

다음 주 예고

Week 4. 변수와 반복 [대면]. 오늘 겪은 불편이 그대로 주제가 됩니다.

오늘 본 것	Week4에서 문제가 되는 지점
<code>backend.tf</code> 에 버킷 이름을 직접 적었습니다	backend는 변수를 못 받습니다. 나머지 값은 어디까지 분리할 수 있을까요
<code>project_name</code> 하나로 리소스 12개 이름을 조립했습니다	이름 목록 자체가 여러 개라면 (<code>for_each</code> vs <code>count</code>)
같은 조립식을 여러 파일에 반복했습니다	한 곳에 모아 관리할 방법이 필요합니다 (<code>locals</code>)
output을 사람이 읽어 옮겨 적었습니다	output 문법을 본격적으로 다룹니다 (<code>output</code> , <code>sensitive</code>)
<code>t3.micro</code> · <code>10.0.1.0/24</code> 가 코드에 직접 적혀 있습니다	하드코딩된 값을 variables로 분리합니다

예습: `variable` 타입(map/list/object), `for_each` vs `count` , `locals` , `output`

부록. 트러블슈팅

증상	원인	해결
<code>plan</code> 이 <code>project_name</code> 을 물어봄	<code>terraform.tfvars</code> 없음	<code>cp example.tfvars terraform.tfvars</code> 후 값 채우기
<code>InvalidBucketName</code>	<code>project_name</code> 에 대문자	소문자·숫자·하이픈만
<code>BucketAlreadyExists</code>	남이 쓰는 이름	<code>project_name</code> 에 본인 GitHub ID 넣기
<code>NoSuchBucket</code> on <code>init</code>	backend의 버킷 이름 오타	A-6 출력과 대조
<code>AccessDenied</code> on <code>init</code>	IAM 권한 부족	개념워크북 Part 3의 권한 목록
migrate 후 <code>7 to add</code>	state가 안 옮겨짐	멈추고 질문. apply 금지
<code>Error acquiring the state lock</code>	잠금이 남음	아무도 작업 안 하는 것 확인 후 <code>force-unlock</code>
backend에 <code>var.</code> 못 씀	init 시점에 변수 미평가	값 직접 입력 또는 <code>-backend-config</code>
<code>destroy</code> 가 SG/서브넷에서 멈춤	ENI 잔존	잠시 후 재실행
<code>Warning: Deprecated Parameter</code>	<code>dynamodb_table</code> 사용	정상입니다. 개념워크북 9번

참고 문서

- [S3 backend](#) : B-4 · B-5 · B-9
- [terraform init](#) : B-5의 `-migrate-state` , B-8의 `-backend-config`

- [State Locking](#) : B-7
- [terraform force-unlock](#) : 부록
- [S3 버전 관리 삭제 \(AWS 문서\)](#) : C-5

이 문서의 리소스 개수와 명령 순서는 `practice/` · `solution/` 코드에서 직접 확인한 값이고, deprecation 경고 문구는 Terraform 1.15.8 · AWS provider 6.58.0 에서 재현한 것입니다. 비용은 개념워크북 Part 6의 조회값과 같습니다.

체크포인트

끝났으면 `app` 스택을 실제로 destroy했는지, 그리고 `bootstrap` 은 변경하지 않았는지 한 번 더 확인하세요. EC2가 켜져 있으면 시간당 \$0.019가 계속 발생하고, 잘못된 순서로 bootstrap을 지우면 다음 주에 처음부터 다시 만들어야 합니다.